Dataframe e Serie di *pandas*

> (c) 2025 Antonio Michele Piemontese

Il package *pandas* di Python serve a gestire i **dataframe** e le **serie**.

```
import pandas as pd     # importazione in memoria del package pandas (già installato di default in Google Colab);
                        # non confondere la installazione di un package con la import: la prima installa SU DISCO e si fa un
                        # sono già pre-installati); la seconda carica il package IN MEMORIA e dunque va fatta ad ogni nuova
```

```
df = pd.read_csv(filepath_or_buffer='Credit_ISLR.csv', sep=",")   # questa riga dà errore --> occorre fare l'upload del file
```

Il file ***Credit*** è un famoso file bancario che contiene 400 clienti di carte di credito descritti lungo una decina di attributi. E' un file simulato negli anni '90 ed è molto usato per imparare python e pandas.

```
df.head()               # le prime 5 righe
```

	Unnamed: 0	ID	Income	Limit	Rating	Cards	Age	Education	Gender	Student	Married	Ethnicity	Balance
0	1	1	14.891	3606	283	2	34	11	Male	No	Yes	Caucasian	333
1	2	2	106.025	6645	483	3	82	15	Female	Yes	Yes	Asian	903
2	3	3	104.593	7075	514	4	71	11	Male	No	No	Asian	580
3	4	4	148.924	9504	681	3	36	11	Female	No	No	Asian	964
4	5	5	55.882	4897	357	2	68	16	Male	No	Yes	Caucasian	331

```
df.tail()               # le ultime 5 righe
```

	Unnamed: 0	ID	Income	Limit	Rating	Cards	Age	Education	Gender	Student	Married	Ethnicity	Balance
395	396	396	12.096	4100	307	3	32	13	Male	No	Yes	Caucasian	560
396	397	397	13.364	3838	296	5	65	17	Male	No	No	African American	480
397	398	398	57.872	4171	321	5	67	12	Female	No	Yes	Caucasian	138
398	399	399	37.728	2525	192	1	44	13	Male	No	Yes	Caucasian	0
399	400	400	18.701	5524	415	5	64	7	Female	No	No	Asian	966

---

pandas fornisce un insieme di funzioni chiamate **read_XXXX** per leggere file di molti formati. Vedi in proposito [questa chat](#).

Rimuoviamo dal dataframe 'df' le colonne inutili:

- *Unnamed: 0*: inserita dalla funzione 'pd.read_csv'
- *ID*: un identificativo numerico del cliente presente nel file CSV. Numera da 1 anzichè da 0, come deve essere in Python; inoltre è ridondante rispetto all'indice che pandas crea in automatico (la prima colonna a sx in grassetto).

```
df.drop(columns=['Unnamed: 0','ID'],inplace=True)  # in-place = True rende l'operazione di drop PERSISTENTE (in memoria)
```

```
display(df)       # display è una FUNZIONE, non un metodo
```

	Income	Limit	Rating	Cards	Age	Education	Gender	Student	Married	Ethnicity	Balance
0	14.891	3606	283	2	34	11	Male	No	Yes	Caucasian	333
1	106.025	6645	483	3	82	15	Female	Yes	Yes	Asian	903
2	104.593	7075	514	4	71	11	Male	No	No	Asian	580
3	148.924	9504	681	3	36	11	Female	No	No	Asian	964
4	55.882	4897	357	2	68	16	Male	No	Yes	Caucasian	331
...	...	...	...	...	...	...	...	...	...	...	...
395	12.096	4100	307	3	32	13	Male	No	Yes	Caucasian	560
396	13.364	3838	296	5	65	17	Male	No	No	African American	480
397	57.872	4171	321	5	67	12	Female	No	Yes	Caucasian	138
398	37.728	2525	192	1	44	13	Male	No	Yes	Caucasian	0
399	18.701	5524	415	5	64	7	Female	No	No	Asian	966

400 rows × 11 columns

La funzione 'display' visualizza le righe e colonne di un dataframe in formato **HTML / CSS** (a differenza della funzione 'print').

Come si vede, le due colonne "droppate" non sono più presenti nel dataframe.

Per vedere **tutte** le righe di un dataframe dobbiamo **settare alcune impostazioni** di pandas (**solo per questo output**):

```
with pd.option_context("display.max_rows", None, "display.max_columns", None):
    display(df)
```

	Income	Limit	Rating	Cards	Age	Education	Gender	Student	Married	Ethnicity	Balance
0	14.891	3606	283	2	34	11	Male	No	Yes	Caucasian	333
1	106.025	6645	483	3	82	15	Female	Yes	Yes	Asian	903
2	104.593	7075	514	4	71	11	Male	No	No	Asian	580
3	148.924	9504	681	3	36	11	Female	No	No	Asian	964
4	55.882	4897	357	2	68	16	Male	No	Yes	Caucasian	331
5	80.180	8047	569	4	77	10	Male	No	No	Caucasian	1151
6	20.996	3388	259	2	37	12	Female	No	No	African American	203
7	71.408	7114	512	2	87	9	Male	No	No	Asian	872
8	15.125	3300	266	5	66	13	Female	No	No	Caucasian	279
9	71.061	6819	491	3	41	19	Female	Yes	Yes	African American	1350
10	63.095	8117	589	4	30	14	Male	No	Yes	Caucasian	1407
11	15.045	1311	138	3	64	16	Male	No	No	Caucasian	0
12	80.616	5308	394	1	57	7	Female	No	Yes	Asian	204
13	43.682	6922	511	1	49	9	Male	No	Yes	Caucasian	1081
14	19.144	3291	269	2	75	13	Female	No	No	African American	148
15	20.089	2525	200	3	57	15	Female	No	Yes	African American	0
16	53.598	3714	286	3	73	17	Female	No	Yes	African American	0
17	36.496	4378	339	3	69	15	Female	No	Yes	Asian	368
18	49.570	6384	448	1	28	9	Female	No	Yes	Asian	891
19	42.079	6626	479	2	44	9	Male	No	No	Asian	1048
20	17.700	2860	235	4	63	16	Female	No	No	Asian	89
21	37.348	6378	458	1	72	17	Female	No	No	Caucasian	968
22	20.103	2631	213	3	61	10	Male	No	Yes	African American	0
23	64.027	5179	398	5	48	8	Male	No	Yes	African American	411
24	10.742	1757	156	3	57	15	Female	No	No	Caucasian	0
25	14.090	4323	326	5	25	16	Female	No	Yes	African American	671
26	42.471	3625	289	6	44	12	Female	Yes	No	Caucasian	654
27	32.793	4534	333	2	44	16	Male	No	No	African American	467
28	186.634	13414	949	2	41	14	Female	No	Yes	African American	1809
29	26.813	5611	411	4	55	16	Female	No	No	Caucasian	915
30	34.142	5666	413	4	47	5	Female	No	Yes	Caucasian	863
31	28.941	2733	210	5	43	16	Male	No	Yes	Asian	0
32	134.181	7838	563	2	48	13	Female	No	No	Caucasian	526
33	31.367	1829	162	4	30	10	Male	No	Yes	Caucasian	0
34	20.150	2646	199	2	25	14	Female	No	Yes	Asian	0
35	23.350	2558	220	3	49	12	Female	Yes	No	Caucasian	419
36	62.413	6457	455	2	71	11	Female	No	Yes	Caucasian	762
37	30.007	6481	462	2	69	9	Female	No	Yes	Caucasian	1093
38	11.795	3899	300	4	25	10	Female	No	No	Caucasian	531
39	13.647	3461	264	4	47	14	Male	No	Yes	Caucasian	344
40	34.950	3327	253	3	54	14	Female	No	No	African American	50
41	113.659	7659	538	2	66	15	Male	Yes	Yes	African American	1155
42	44.158	4763	351	2	66	13	Female	No	Yes	Asian	385
43	36.929	6257	445	1	24	14	Female	No	Yes	Asian	976
44	31.861	6375	469	3	25	16	Female	No	Yes	Caucasian	1120
45	77.380	7569	564	3	50	12	Female	No	Yes	Caucasian	997
46	19.531	5043	376	2	64	16	Female	Yes	Yes	Asian	1241
47	44.646	4431	320	2	49	15	Male	Yes	Yes	Caucasian	797
48	44.522	2252	205	6	72	15	Male	No	Yes	Asian	0

49	43.479	4569	354	4	49	13	Male	Yes	Yes	African American	902
50	36.362	5183	376	3	49	15	Male	No	Yes	African American	654
51	39.705	3969	301	2	27	20	Male	No	Yes	African American	211
52	44.205	5441	394	1	32	12	Male	No	Yes	Caucasian	607
53	16.304	5466	413	4	66	10	Male	No	Yes	Asian	957
54	15.333	1499	138	2	47	9	Female	No	Yes	Asian	0
55	32.916	1786	154	2	60	8	Female	No	Yes	Asian	0
56	57.100	4742	372	7	79	18	Female	No	Yes	Asian	379
57	76.273	4779	367	4	65	14	Female	No	Yes	Caucasian	133
58	10.354	3480	281	2	70	17	Male	No	Yes	Caucasian	333
59	51.872	5294	390	4	81	17	Female	No	No	Caucasian	531
60	35.510	5198	364	2	35	20	Female	No	No	Asian	631
61	21.238	3089	254	3	59	10	Female	No	No	Caucasian	108
62	30.682	1671	160	2	77	7	Female	No	No	Caucasian	0
63	14.132	2998	251	4	75	17	Male	No	No	Caucasian	133
64	32.164	2937	223	2	79	15	Female	No	Yes	African American	0
65	12.000	4160	320	4	28	14	Female	No	Yes	Caucasian	602
66	113.829	9704	694	4	38	13	Female	No	Yes	Asian	1388
67	11.187	5099	380	4	69	16	Female	No	No	African American	889
68	27.847	5619	418	2	78	15	Female	No	Yes	Caucasian	822
69	49.502	6819	505	4	55	14	Male	No	Yes	Caucasian	1084
70	24.889	3954	318	4	75	12	Male	No	Yes	Caucasian	357
71	58.781	7402	538	2	81	12	Female	No	Yes	Asian	1103
72	22.939	4923	355	1	47	18	Female	No	Yes	Asian	663
73	23.989	4523	338	4	31	15	Male	No	No	Caucasian	601
74	16.103	5390	418	4	45	10	Female	No	Yes	Caucasian	945
75	33.017	3180	224	2	28	16	Male	No	Yes	African American	29
76	30.622	3293	251	1	68	16	Male	Yes	No	Caucasian	532
77	20.936	3254	253	1	30	15	Female	No	No	Asian	145
78	110.968	6662	468	3	45	11	Female	No	Yes	Caucasian	391
79	15.354	2101	171	2	65	14	Male	No	No	Asian	0
80	27.369	3449	288	3	40	9	Female	No	Yes	Caucasian	162
81	53.480	4263	317	1	83	15	Male	No	No	Caucasian	99
82	23.672	4433	344	3	63	11	Male	No	No	Caucasian	503
83	19.225	1433	122	3	38	14	Female	No	No	Caucasian	0
84	43.540	2906	232	4	69	11	Male	No	No	Caucasian	0
85	152.298	12066	828	4	41	12	Female	No	Yes	Asian	1779
86	55.367	6340	448	1	33	15	Male	No	Yes	Caucasian	815
87	11.741	2271	182	4	59	12	Female	No	No	Asian	0
88	15.560	4307	352	4	57	8	Male	No	Yes	African American	579
89	59.530	7518	543	3	52	9	Female	No	No	African American	1176
90	20.191	5767	431	4	42	16	Male	No	Yes	African American	1023
91	48.498	6040	456	3	47	16	Male	No	Yes	Caucasian	812
92	30.733	2832	249	4	51	13	Male	No	No	Caucasian	0
93	16.479	5435	388	2	26	16	Male	No	No	African American	937
94	38.009	3075	245	3	45	15	Female	No	No	African American	0
95	14.084	855	120	5	46	17	Female	No	Yes	African American	0
96	14.312	5382	367	1	59	17	Male	Yes	No	Asian	1380
97	26.067	3388	266	4	74	17	Female	No	Yes	African American	155
98	36.295	2963	241	2	68	14	Female	Yes	No	African American	375

99	83.851	8494	607	5	47	18	Male	No	No	Caucasian	1311
100	21.153	3736	256	1	41	11	Male	No	No	Caucasian	298
101	17.976	2433	190	3	70	16	Female	Yes	No	Caucasian	431
102	68.713	7582	531	2	56	16	Male	Yes	No	Caucasian	1587
103	146.183	9540	682	6	66	15	Male	No	No	Caucasian	1050
104	15.846	4768	365	4	53	12	Female	No	No	Caucasian	745
105	12.031	3182	259	2	58	18	Female	No	Yes	Caucasian	210
106	16.819	1337	115	2	74	15	Male	No	Yes	Asian	0
107	39.110	3189	263	3	72	12	Male	No	No	Asian	0
108	107.986	6033	449	4	64	14	Male	No	Yes	Caucasian	227
109	13.561	3261	279	5	37	19	Male	No	Yes	Asian	297
110	34.537	3271	250	3	57	17	Female	No	Yes	Asian	47
111	28.575	2959	231	2	60	11	Female	No	No	African American	0
112	46.007	6637	491	4	42	14	Male	No	Yes	Caucasian	1046
113	69.251	6386	474	4	30	12	Female	No	Yes	Asian	768
114	16.482	3326	268	4	41	15	Male	No	No	Caucasian	271
115	40.442	4828	369	5	81	8	Female	No	No	African American	510
116	35.177	2117	186	3	62	16	Female	No	No	Caucasian	0
117	91.362	9113	626	1	47	17	Male	No	Yes	Asian	1341
118	27.039	2161	173	3	40	17	Female	No	No	Caucasian	0
119	23.012	1410	137	3	81	16	Male	No	No	Caucasian	0
120	27.241	1402	128	2	67	15	Female	No	Yes	Asian	0
121	148.080	8157	599	2	83	13	Male	No	Yes	Caucasian	454
122	62.602	7056	481	1	84	11	Female	No	No	Caucasian	904
123	11.808	1300	117	3	77	14	Female	No	No	African American	0
124	29.564	2529	192	1	30	12	Female	No	Yes	Caucasian	0
125	27.578	2531	195	1	34	15	Female	No	Yes	Caucasian	0
126	26.427	5533	433	5	50	15	Female	Yes	Yes	Asian	1404
127	57.202	3411	259	3	72	11	Female	No	No	Caucasian	0
128	123.299	8376	610	2	89	17	Male	Yes	No	African American	1259
129	18.145	3461	279	3	56	15	Male	No	Yes	African American	255
130	23.793	3821	281	4	56	12	Female	Yes	Yes	African American	868
131	10.726	1568	162	5	46	19	Male	No	Yes	Asian	0
132	23.283	5443	407	4	49	13	Male	No	Yes	African American	912
133	21.455	5829	427	4	80	12	Female	No	Yes	African American	1018
134	34.664	5835	452	3	77	15	Female	No	Yes	African American	835
135	44.473	3500	257	3	81	16	Female	No	No	African American	8
136	54.663	4116	314	2	70	8	Female	No	No	African American	75
137	36.355	3613	278	4	35	9	Male	No	Yes	Asian	187
138	21.374	2073	175	2	74	11	Female	No	Yes	Caucasian	0
139	107.841	10384	728	3	87	7	Male	No	No	African American	1597
140	39.831	6045	459	3	32	12	Female	Yes	Yes	African American	1425
141	91.876	6754	483	2	33	10	Male	No	Yes	Caucasian	605
142	103.893	7416	549	3	84	17	Male	No	No	Asian	669
143	19.636	4896	387	3	64	10	Female	No	No	African American	710
144	17.392	2748	228	3	32	14	Male	No	Yes	Caucasian	68
145	19.529	4673	341	2	51	14	Male	No	No	Asian	642
146	17.055	5110	371	3	55	15	Female	No	Yes	Caucasian	805
147	23.857	1501	150	3	56	16	Male	No	Yes	Caucasian	0
148	15.184	2420	192	2	69	11	Female	No	Yes	Caucasian	0

149	13.444	886	121	5	44	10	Male	No	Yes	Asian	0
150	63.931	5728	435	3	28	14	Female	No	Yes	African American	581
151	35.864	4831	353	3	66	13	Female	No	Yes	Caucasian	534
152	41.419	2120	184	4	24	11	Female	Yes	No	Caucasian	156
153	92.112	4612	344	3	32	17	Male	No	No	Caucasian	0
154	55.056	3155	235	2	31	16	Male	No	Yes	African American	0
155	19.537	1362	143	4	34	9	Female	No	Yes	Asian	0
156	31.811	4284	338	5	75	13	Female	No	Yes	Caucasian	429
157	56.256	5521	406	2	72	16	Female	Yes	Yes	Caucasian	1020
158	42.357	5550	406	2	83	12	Female	No	Yes	Asian	653
159	53.319	3000	235	3	53	13	Male	No	No	Asian	0
160	12.238	4865	381	5	67	11	Female	No	No	Caucasian	836
161	31.353	1705	160	3	81	14	Male	No	Yes	Caucasian	0
162	63.809	7530	515	1	56	12	Male	No	Yes	Caucasian	1086
163	13.676	2330	203	5	80	16	Female	No	No	African American	0
164	76.782	5977	429	4	44	12	Male	No	Yes	Asian	548
165	25.383	4527	367	4	46	11	Male	No	Yes	Caucasian	570
166	35.691	2880	214	2	35	15	Male	No	No	African American	0
167	29.403	2327	178	1	37	14	Female	No	Yes	Caucasian	0
168	27.470	2820	219	1	32	11	Female	No	Yes	Asian	0
169	27.330	6179	459	4	36	12	Female	No	Yes	Caucasian	1099
170	34.772	2021	167	3	57	9	Male	No	No	Asian	0
171	36.934	4270	299	1	63	9	Female	No	Yes	Caucasian	283
172	76.348	4697	344	4	60	18	Male	No	No	Asian	108
173	14.887	4745	339	3	58	12	Male	No	Yes	African American	724
174	121.834	10673	750	3	54	16	Male	No	No	African American	1573
175	30.132	2168	206	3	52	17	Male	No	No	Caucasian	0
176	24.050	2607	221	4	32	18	Male	No	Yes	Caucasian	0
177	22.379	3965	292	2	34	14	Female	No	Yes	Asian	384
178	28.316	4391	316	2	29	10	Female	No	No	Caucasian	453
179	58.026	7499	560	5	67	11	Female	No	No	Caucasian	1237
180	10.635	3584	294	5	69	16	Male	No	Yes	Asian	423
181	46.102	5180	382	3	81	12	Male	No	Yes	African American	516
182	58.929	6420	459	2	66	9	Female	No	Yes	African American	789
183	80.861	4090	335	3	29	15	Female	No	Yes	Asian	0
184	158.889	11589	805	1	62	17	Female	No	Yes	Caucasian	1448
185	30.420	4442	316	1	30	14	Female	No	No	African American	450
186	36.472	3806	309	2	52	13	Male	No	No	African American	188
187	23.365	2179	167	2	75	15	Male	No	No	Asian	0
188	83.869	7667	554	2	83	11	Male	No	No	African American	930
189	58.351	4411	326	2	85	16	Female	No	Yes	Caucasian	126
190	55.187	5352	385	4	50	17	Female	No	Yes	Caucasian	538
191	124.290	9560	701	3	52	17	Female	Yes	No	Asian	1687
192	28.508	3933	287	4	56	14	Male	No	Yes	Asian	336
193	130.209	10088	730	7	39	19	Female	No	Yes	Caucasian	1426
194	30.406	2120	181	2	79	14	Male	No	Yes	African American	0
195	23.883	5384	398	2	73	16	Female	No	Yes	African American	802
196	93.039	7398	517	1	67	12	Male	No	Yes	African American	749
197	50.699	3977	304	2	84	17	Female	No	No	African American	69
198	27.349	2000	169	4	51	16	Female	No	Yes	African American	0

199	10.403	4159	310	3	43	7	Male	No	Yes	Asian	571
200	23.949	5343	383	2	40	18	Male	No	Yes	African American	829
201	73.914	7333	529	6	67	15	Female	No	Yes	Caucasian	1048
202	21.038	1448	145	2	58	13	Female	No	Yes	Caucasian	0
203	68.206	6784	499	5	40	16	Female	Yes	No	African American	1411
204	57.337	5310	392	2	45	7	Female	No	No	Caucasian	456
205	10.793	3878	321	8	29	13	Male	No	No	Caucasian	638
206	23.450	2450	180	2	78	13	Male	No	No	Caucasian	0
207	10.842	4391	358	5	37	10	Female	Yes	Yes	Caucasian	1216
208	51.345	4327	320	3	46	15	Male	No	No	African American	230
209	151.947	9156	642	2	91	11	Female	No	Yes	African American	732
210	24.543	3206	243	2	62	12	Female	No	Yes	Caucasian	95
211	29.567	5309	397	3	25	15	Male	No	No	Caucasian	799
212	39.145	4351	323	2	66	13	Male	No	Yes	Caucasian	308
213	39.422	5245	383	2	44	19	Male	No	No	African American	637
214	34.909	5289	410	2	62	16	Female	No	Yes	Caucasian	681
215	41.025	4229	337	3	79	19	Female	No	Yes	Caucasian	246
216	15.476	2762	215	3	60	18	Male	No	No	Asian	52
217	12.456	5395	392	3	65	14	Male	No	Yes	Caucasian	955
218	10.627	1647	149	2	71	10	Female	Yes	Yes	Asian	195
219	38.954	5222	370	4	76	13	Female	No	No	Caucasian	653
220	44.847	5765	437	3	53	13	Female	Yes	No	Asian	1246
221	98.515	8760	633	5	78	11	Female	No	No	African American	1230
222	33.437	6207	451	4	44	9	Male	Yes	No	Caucasian	1549
223	27.512	4613	344	5	72	17	Male	No	Yes	Asian	573
224	121.709	7818	584	4	50	6	Male	No	Yes	Caucasian	701
225	15.079	5673	411	4	28	15	Female	No	Yes	Asian	1075
226	59.879	6906	527	6	78	15	Female	No	No	Caucasian	1032
227	66.989	5614	430	3	47	14	Female	No	Yes	Caucasian	482
228	69.165	4668	341	2	34	11	Female	No	No	African American	156
229	69.943	7555	547	3	76	9	Male	No	Yes	Asian	1058
230	33.214	5137	387	3	59	9	Male	No	No	African American	661
231	25.124	4776	378	4	29	12	Male	No	Yes	Caucasian	657
232	15.741	4788	360	1	39	14	Male	No	Yes	Asian	689
233	11.603	2278	187	3	71	11	Male	No	Yes	Caucasian	0
234	69.656	8244	579	3	41	14	Male	No	Yes	African American	1329
235	10.503	2923	232	3	25	18	Female	No	Yes	African American	191
236	42.529	4986	369	2	37	11	Male	No	Yes	Asian	489
237	60.579	5149	388	5	38	15	Male	No	Yes	Asian	443
238	26.532	2910	236	6	58	19	Female	No	Yes	Caucasian	52
239	27.952	3557	263	1	35	13	Female	No	Yes	Asian	163
240	29.705	3351	262	5	71	14	Female	No	Yes	Asian	148
241	15.602	906	103	2	36	11	Male	No	Yes	African American	0
242	20.918	1233	128	3	47	18	Female	Yes	Yes	Asian	16
243	58.165	6617	460	1	56	12	Female	No	Yes	Caucasian	856
244	22.561	1787	147	4	66	15	Female	No	No	Caucasian	0
245	34.509	2001	189	5	80	18	Female	No	Yes	African American	0
246	19.588	3211	265	4	59	14	Female	No	No	Asian	199
247	36.364	2220	188	3	50	19	Male	No	No	Caucasian	0
248	15.717	905	93	1	38	16	Male	Yes	Yes	Caucasian	0

249	22.574	1551	134	3	43	13	Female	Yes	Yes	Caucasian	98
250	10.363	2430	191	2	47	18	Female	No	Yes	Asian	0
251	28.474	3202	267	5	66	12	Male	No	Yes	Caucasian	132
252	72.945	8603	621	3	64	8	Female	No	No	Caucasian	1355
253	85.425	5182	402	6	60	12	Male	No	Yes	African American	218
254	36.508	6386	469	4	79	6	Female	No	Yes	Caucasian	1048
255	58.063	4221	304	3	50	8	Male	No	No	African American	118
256	25.936	1774	135	2	71	14	Female	No	No	Asian	0
257	15.629	2493	186	1	60	14	Male	No	Yes	Asian	0
258	41.400	2561	215	2	36	14	Male	No	Yes	Caucasian	0
259	33.657	6196	450	6	55	9	Female	No	No	Caucasian	1092
260	67.937	5184	383	4	63	12	Male	No	Yes	Asian	345
261	180.379	9310	665	3	67	8	Female	Yes	Yes	Asian	1050
262	10.588	4049	296	1	66	13	Female	No	Yes	Caucasian	465
263	29.725	3536	270	2	52	15	Female	No	No	African American	133
264	27.999	5107	380	1	55	10	Male	No	Yes	Caucasian	651
265	40.885	5013	379	3	46	13	Female	No	Yes	African American	549
266	88.830	4952	360	4	86	16	Female	No	Yes	Caucasian	15
267	29.638	5833	433	3	29	15	Female	No	Yes	Asian	942
268	25.988	1349	142	4	82	12	Male	No	No	Caucasian	0
269	39.055	5565	410	4	48	18	Female	No	Yes	Caucasian	772
270	15.866	3085	217	1	39	13	Male	No	No	Caucasian	136
271	44.978	4866	347	1	30	10	Female	No	No	Caucasian	436
272	30.413	3690	299	2	25	15	Female	Yes	No	Asian	728
273	16.751	4706	353	6	48	14	Male	Yes	No	Asian	1255
274	30.550	5869	439	5	81	9	Female	No	No	African American	967
275	163.329	8732	636	3	50	14	Male	No	Yes	Caucasian	529
276	23.106	3476	257	2	50	15	Female	No	No	Caucasian	209
277	41.532	5000	353	2	50	12	Male	No	Yes	Caucasian	531
278	128.040	6982	518	2	78	11	Female	No	Yes	Caucasian	250
279	54.319	3063	248	3	59	8	Female	Yes	No	Caucasian	269
280	53.401	5319	377	3	35	12	Female	No	No	African American	541
281	36.142	1852	183	3	33	13	Female	No	No	African American	0
282	63.534	8100	581	2	50	17	Female	No	Yes	Caucasian	1298
283	49.927	6396	485	3	75	17	Female	No	Yes	Caucasian	890
284	14.711	2047	167	2	67	6	Male	No	Yes	Caucasian	0
285	18.967	1626	156	2	41	11	Female	No	Yes	Asian	0
286	18.036	1552	142	2	48	15	Female	No	No	Caucasian	0
287	60.449	3098	272	4	69	8	Male	No	Yes	Caucasian	0
288	16.711	5274	387	3	42	16	Female	No	Yes	Asian	863
289	10.852	3907	296	2	30	9	Male	No	No	Caucasian	485
290	26.370	3235	268	5	78	11	Male	No	Yes	Asian	159
291	24.088	3665	287	4	56	13	Female	No	Yes	Caucasian	309
292	51.532	5096	380	2	31	15	Male	No	Yes	Caucasian	481
293	140.672	11200	817	7	46	9	Male	No	Yes	African American	1677
294	42.915	2532	205	4	42	13	Male	No	Yes	Asian	0
295	27.272	1389	149	5	67	10	Female	No	Yes	Caucasian	0
296	65.896	5140	370	1	49	17	Female	No	Yes	Caucasian	293
297	55.054	4381	321	3	74	17	Male	No	Yes	Asian	188
298	20.791	2672	204	1	70	18	Female	No	No	African American	0

299	24.919	5051	372	3	76	11	Female	No	Yes	African American	711
300	21.786	4632	355	1	50	17	Male	No	Yes	Caucasian	580
301	31.335	3526	289	3	38	7	Female	No	No	Caucasian	172
302	59.855	4964	365	1	46	13	Female	No	Yes	Caucasian	295
303	44.061	4970	352	1	79	11	Male	No	Yes	African American	414
304	82.706	7506	536	2	64	13	Female	No	Yes	Asian	905
305	24.460	1924	165	2	50	14	Female	No	Yes	Asian	0
306	45.120	3762	287	3	80	8	Male	No	Yes	Caucasian	70
307	75.406	3874	298	3	41	14	Female	No	Yes	Asian	0
308	14.956	4640	332	2	33	6	Male	No	No	Asian	681
309	75.257	7010	494	3	34	18	Female	No	Yes	Caucasian	885
310	33.694	4891	369	1	52	16	Male	Yes	No	African American	1036
311	23.375	5429	396	3	57	15	Female	No	Yes	Caucasian	844
312	27.825	5227	386	6	63	11	Male	No	Yes	Caucasian	823
313	92.386	7685	534	2	75	18	Female	No	Yes	Asian	843
314	115.520	9272	656	2	69	14	Male	No	No	African American	1140
315	14.479	3907	296	3	43	16	Male	No	Yes	Caucasian	463
316	52.179	7306	522	2	57	14	Male	No	No	Asian	1142
317	68.462	4712	340	2	71	16	Male	No	Yes	Caucasian	136
318	18.951	1485	129	3	82	13	Female	No	No	Caucasian	0
319	27.590	2586	229	5	54	16	Male	No	Yes	African American	0
320	16.279	1160	126	3	78	13	Male	Yes	Yes	African American	5
321	25.078	3096	236	2	27	15	Female	No	Yes	Caucasian	81
322	27.229	3484	282	6	51	11	Male	No	No	Caucasian	265
323	182.728	13913	982	4	98	17	Male	No	Yes	Caucasian	1999
324	31.029	2863	223	2	66	17	Male	Yes	Yes	Asian	415
325	17.765	5072	364	1	66	12	Female	No	Yes	Caucasian	732
326	125.480	10230	721	3	82	16	Male	No	Yes	Caucasian	1361
327	49.166	6662	508	3	68	14	Female	No	No	Asian	984
328	41.192	3673	297	3	54	16	Female	No	Yes	Caucasian	121
329	94.193	7576	527	2	44	16	Female	No	Yes	Caucasian	846
330	20.405	4543	329	2	72	17	Male	Yes	No	Asian	1054
331	12.581	3976	291	2	48	16	Male	No	Yes	Caucasian	474
332	62.328	5228	377	3	83	15	Male	No	No	Caucasian	380
333	21.011	3402	261	2	68	17	Male	No	Yes	African American	182
334	24.230	4756	351	2	64	15	Female	No	Yes	Caucasian	594
335	24.314	3409	270	2	23	7	Female	No	Yes	Caucasian	194
336	32.856	5884	438	4	68	13	Male	No	No	Caucasian	926
337	12.414	855	119	3	32	12	Male	No	Yes	African American	0
338	41.365	5303	377	1	45	14	Male	No	No	Caucasian	606
339	149.316	10278	707	1	80	16	Male	No	No	African American	1107
340	27.794	3807	301	4	35	8	Female	No	Yes	African American	320
341	13.234	3922	299	2	77	17	Female	No	Yes	Caucasian	426
342	14.595	2955	260	5	37	9	Male	No	Yes	African American	204
343	10.735	3746	280	2	44	17	Female	No	Yes	Caucasian	410
344	48.218	5199	401	7	39	10	Male	No	Yes	Asian	633
345	30.012	1511	137	2	33	17	Male	No	Yes	Caucasian	0
346	21.551	5380	420	5	51	18	Male	No	Yes	Asian	907
347	160.231	10748	754	2	69	17	Male	No	No	Caucasian	1192
348	13.433	1134	112	3	70	14	Male	No	Yes	Caucasian	0

349	48.577	5145	389	3	71	13	Female	No	Yes	Asian	503
350	30.002	1561	155	4	70	13	Female	No	Yes	Caucasian	0
351	61.620	5140	374	1	71	9	Male	No	Yes	Caucasian	302
352	104.483	7140	507	2	41	14	Male	No	Yes	African American	583
353	41.868	4716	342	2	47	18	Male	No	No	Caucasian	425
354	12.068	3873	292	1	44	18	Female	No	Yes	Asian	413
355	180.682	11966	832	2	58	8	Female	No	Yes	African American	1405
356	34.480	6090	442	3	36	14	Male	No	No	Caucasian	962
357	39.609	2539	188	1	40	14	Male	No	Yes	Asian	0
358	30.111	4336	339	1	81	18	Male	No	Yes	Caucasian	347
359	12.335	4471	344	3	79	12	Male	No	Yes	African American	611
360	53.566	5891	434	4	82	10	Female	No	No	Caucasian	712
361	53.217	4943	362	2	46	16	Female	No	Yes	Asian	382
362	26.162	5101	382	3	62	19	Female	No	No	African American	710
363	64.173	6127	433	1	80	10	Male	No	Yes	Caucasian	578
364	128.669	9824	685	3	67	16	Male	No	Yes	Asian	1243
365	113.772	6442	489	4	69	15	Male	Yes	Yes	Caucasian	790
366	61.069	7871	564	3	56	14	Male	No	Yes	Caucasian	1264
367	23.793	3615	263	2	70	14	Male	No	No	African American	216
368	89.000	5759	440	3	37	6	Female	No	No	Caucasian	345
369	71.682	8028	599	3	57	16	Male	No	Yes	Caucasian	1208
370	35.610	6135	466	4	40	12	Male	No	No	Caucasian	992
371	39.116	2150	173	4	75	15	Male	No	No	Caucasian	0
372	19.782	3782	293	2	46	16	Female	Yes	No	Caucasian	840
373	55.412	5354	383	2	37	16	Female	Yes	Yes	Caucasian	1003
374	29.400	4840	368	3	76	18	Female	No	Yes	Caucasian	588
375	20.974	5673	413	5	44	16	Female	No	Yes	Caucasian	1000
376	87.625	7167	515	2	46	10	Female	No	No	African American	767
377	28.144	1567	142	3	51	10	Male	No	Yes	Caucasian	0
378	19.349	4941	366	1	33	19	Male	No	Yes	Caucasian	717
379	53.308	2860	214	1	84	10	Male	No	Yes	Caucasian	0
380	115.123	7760	538	3	83	14	Female	No	No	African American	661
381	101.788	8029	574	2	84	11	Male	No	Yes	Caucasian	849
382	24.824	5495	409	1	33	9	Male	Yes	No	Caucasian	1352
383	14.292	3274	282	9	64	9	Male	No	Yes	Caucasian	382
384	20.088	1870	180	3	76	16	Male	No	No	African American	0
385	26.400	5640	398	3	58	15	Female	No	No	Asian	905
386	19.253	3683	287	4	57	10	Male	No	No	African American	371
387	16.529	1357	126	3	62	9	Male	No	No	Asian	0
388	37.878	6827	482	2	80	13	Female	No	No	Caucasian	1129
389	83.948	7100	503	2	44	18	Male	No	No	Caucasian	806
390	135.118	10578	747	3	81	15	Female	No	Yes	Asian	1393
391	73.327	6555	472	2	43	15	Female	No	No	Caucasian	721
392	25.974	2308	196	2	24	10	Male	No	No	Asian	0
393	17.316	1335	138	2	65	13	Male	No	No	African American	0
394	49.794	5758	410	4	40	8	Male	No	No	Caucasian	734
395	12.096	4100	307	3	32	13	Male	No	Yes	Caucasian	560
396	13.364	3838	296	5	65	17	Male	No	No	African American	480
397	57.872	4171	321	5	67	12	Female	No	Yes	Caucasian	138
398	37.728	2525	192	1	44	13	Male	No	Yes	Caucasian	0
399	18.701	5524	415	5	64	7	Female	No	No	Asian	966

La **classe** di un oggetto

```
type(df)             # la classe dell'oggetto --> è un dataframe perchè è stato creato con la funzione 'pd.read_csv' di pa
```

```
pandas.DataFrame
```

I datatype elementari (delle singole colonne):

```
df.dtypes
```

```
Income       float64
Limit          int64
Rating         int64
Cards          int64
Age            int64
Education      int64
Gender           str
Student          str
Married          str
Ethnicity        str
Balance        int64
dtype: object
```

Le dimensioni del dataframe con le relative cardinalità:

```
df.shape
```

```
(400, 11)
```

L'occupazione in byte del dataframe:

```
df.size
```

```
4400
```

Cosa è un dataframe?
E' una TABELLA di righe e colonne **in memoria**. Non è la classica tabella SQL (su disco).

Vediamonne le **dimensioni** (con il relativo **size**):

```
df.shape
```

```
(400, 11)
```

Il dataframe 'df' ha 2 dimemnsioni di size 400 e 11.

L'elenco dei data-type pandas **elementari** del 'df' (cioè delle singole colonne) è fornito dal metodo 'dtypes'.

```
df.dtypes
```

```
Income       float64
Limit          int64
Rating         int64
Cards          int64
Age            int64
Education      int64
Gender           str
Student          str
Married          str
Ethnicity        str
Balance        int64
dtype: object
```

Quanto spazio in memoria (in byte) occupa il dataframe?

```
df.size
```

```
4400
```

Il dataframe contiene valori mancanti? (detti **missing values** oppure **not available**). Si fa con il metodo 'isna' che funziona in questo modo:

- lavora in modo parrallelo su righe e colonne

- esegue il test sulla **singola cella**
- e quindi restituisce una matrice di booleani (True oppure False) con l'esito del test per ogni cella.

```
df.isna()
```

	Income	Limit	Rating	Cards	Age	Education	Gender	Student	Married	Ethnicity	Balance
0	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...
395	False	False	False	False	False	False	False	False	False	False	False
396	False	False	False	False	False	False	False	False	False	False	False
397	False	False	False	False	False	False	False	False	False	False	False
398	False	False	False	False	False	False	False	False	False	False	False
399	False	False	False	False	False	False	False	False	False	False	False

400 rows × 11 columns

Occorre individuare le celle con **True** (i MV). Per dataset medio-grandi questa operazione è di fatto impossibile.

Per fortuna, il valore booleano True è memorizzato nei PC come 1, ed il valore False come 0.

```
df.isna().sum()
```

```
Income       0
Limit        0
Rating       0
Cards        0
Age          0
Education    0
Gender       0
Student      0
Married      0
Ethnicity    0
Balance      0
dtype: int64
```

Ora un pò di **analisi** di questo dataset:

```
df.info()          # fornisce una SINTESI di informazioni sul dataframe
```

```
<class 'pandas.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 11 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Income     400 non-null    float64
 1   Limit      400 non-null    int64
 2   Rating     400 non-null    int64
 3   Cards      400 non-null    int64
 4   Age        400 non-null    int64
 5   Education  400 non-null    int64
 6   Gender     400 non-null    str
 7   Student    400 non-null    str
 8   Married    400 non-null    str
 9   Ethnicity  400 non-null    str
 10  Balance    400 non-null    int64
dtypes: float64(1), int64(6), str(4)
memory usage: 42.3 KB
```

`float64` è un numero a virgola mobile in doppia precisione (cioè con i decimali e allocato su 64 bit).
`int64` è un numero intero allocato su 64 bit
`object` è una stringa alfanumerica

Possiamo ottenere alcune di queste info separatamente, con comandi differenti:

```
display(df.columns.tolist())
display(df.dtypes)
```

```
print('\n','numero di NA complessivo: ',df.isna().sum().sum().item(),'\n')
print('byte del size: ', df.size)
```

```
['Income',
 'Limit',
 'Rating',
 'Cards',
 'Age',
 'Education',
 'Gender',
 'Student',
 'Married',
 'Ethnicity',
 'Balance']
Income       float64
Limit          int64
Rating         int64
Cards          int64
Age            int64
Education      int64
Gender           str
Student          str
Married          str
Ethnicity        str
Balance        int64
dtype: object

 numero di NA complessivo:  0

byte del size:  4400
```

```
df.describe()   # calcola le statistiche di base per le SOLE colonne NUMERICHE (int, float, ecc)
```

	Income	Limit	Rating	Cards	Age	Education	Balance
count	400.000000	400.000000	400.000000	400.000000	400.000000	400.000000	400.000000
mean	45.218885	4735.600000	354.940000	2.957500	55.667500	13.450000	520.015000
std	35.244273	2308.198848	154.724143	1.371275	17.249807	3.125207	459.758877
min	10.354000	855.000000	93.000000	1.000000	23.000000	5.000000	0.000000
25%	21.007250	3088.000000	247.250000	2.000000	41.750000	11.000000	68.750000
50%	33.115500	4622.500000	344.000000	3.000000	56.000000	14.000000	459.500000
75%	57.470750	5872.750000	437.250000	4.000000	70.000000	16.000000	863.000000
max	186.634000	13913.000000	982.000000	9.000000	98.000000	20.000000	1999.000000

```
round(df.describe(),2)
```

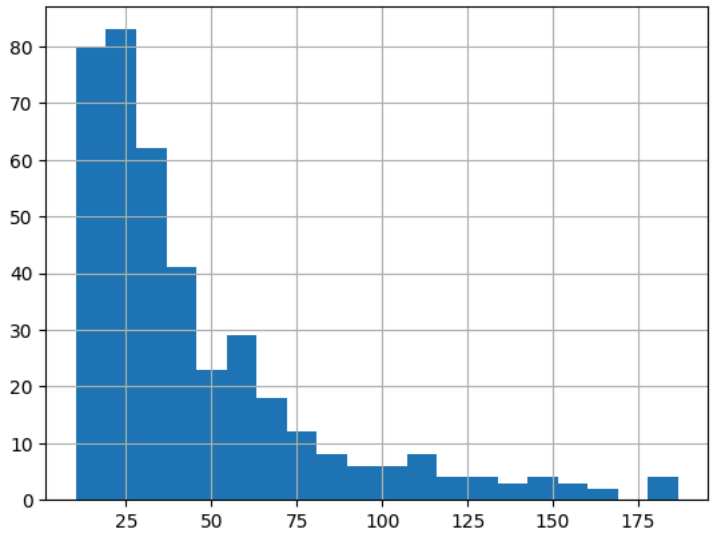
	Income	Limit	Rating	Cards	Age	Education	Balance
count	400.00	400.00	400.00	400.00	400.00	400.00	400.00
mean	45.22	4735.60	354.94	2.96	55.67	13.45	520.02
std	35.24	2308.20	154.72	1.37	17.25	3.13	459.76
min	10.35	855.00	93.00	1.00	23.00	5.00	0.00
25%	21.01	3088.00	247.25	2.00	41.75	11.00	68.75
50%	33.12	4622.50	344.00	3.00	56.00	14.00	459.50
75%	57.47	5872.75	437.25	4.00	70.00	16.00	863.00
max	186.63	13913.00	982.00	9.00	98.00	20.00	1999.00

```
df['Income'].median()    # mediana molto più bassa della media; come mai? indaghiamo...
```

```
33.1155
```

```
df['Income'].hist(bins=20)
```

```
<Axes: >
```



Nota sulla mediana

Ad ottobre 2025 si è discusso molto su questo post Instagram di **StartingFinance**. Vedi [questa *reaction*]().

> Commento di *StartingFinance*:
> Lombardia e Trentino-Alto Adige si confermano le regioni con gli stipendi più alti d'Italia. Nel 2024, la RAL media è di circa €33.600, che corrisponde a €1.960 netti al mese per 13 mensilità. Seguono da vicino Lazio (€33.200) e Liguria (€33.100), mentre le regioni del Sud si fermano tra i €27.000 e i €29.000 lordi annui — come Basilicata, Calabria e Sicilia.
> Questi dati, elaborati dall'Osservatorio JobPricing nel report Salary Outlook 2025, rappresentano valori medi. Significa che esistono differenze significative tra settori, ruoli e livelli di esperienza, ma servono a tracciare un quadro generale della distribuzione del reddito.
> Le RAL regionali permettono infatti di stimare il potere d'acquisto e la competitività dei territori, ma anche di leggere il divario Nord-Sud: oltre €6.000 lordi all'anno separano le prime regioni in classifica dalle ultime.
> Un indicatore imperfetto, certo — ma indispensabile per capire dove il lavoro "paga" di più.

Il metodo 'describe', come molti altri metodi e funzioni di pandas e python, utilizza il famoso **funzionamento VETTORIALE**, nel quale le colonne sono elaborate in modo **parallelo ed indipendente**.

In linguaggi tradizionali, anche ad oggetti come Java, C#, ecc, il metodo 'describe' - se applicato a tutto il dataframe - sarebbe implementato con un ciclo 'for' (una iterazione per colonna).

Il funzionamento vettoriale permette inoltre di scrivere **codice più compatto e leggibile**

```
print(df.shape)
print(df['Income'].shape)
print(type(df['Income']))

print(df[['Income']].shape)
print(type(df[['Income']]))
```

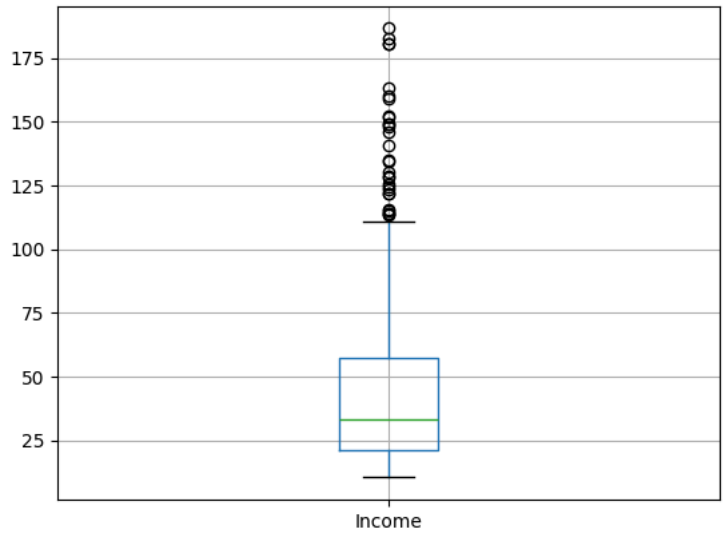
```
(400, 11)
(400,)
<class 'pandas.Series'>
(400, 1)
<class 'pandas.DataFrame'>
```

pandas ha due tipi di data-type aggregati: il dataframe (righe e colonne) e **la serie**, composta da un vettore (serie di elementi).

Queste 2 classi hanno metodi differenti! (a parte alcuni: head, tail, shape, describe, ecc)

```
df[['Income']].boxplot()
```

```
<Axes: >
```



I baffi esprimono l'IQR (Inter Quartile Range), cioè la differenza tra Q3 (75%) e Q1 (25%), moltiplicato per 1.5. E' una convenzione che permette di inviduare eventuali outlier, cioè valori "anomali".

Calcoliamo la fondamentale matrice di correlazione

```
df.corr()    # --> dà errore perchè le correlazioni sono calcolabili solo tra coppie di variabili numeriche
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[28], line 1
----> 1 df.corr()    # --> dà errore perchè le correlazioni sono calcolabili solo tra coppie di variabili numeriche

File c:\Users\Utente\AppData\Local\spyder-6\Lib\site-packages\pandas\core\frame.py:13121, in DataFrame.corr(self, method,
min_periods, numeric_only)
  13117         """  # noqa: E501
  13118         data = self._get_numeric_data() if numeric_only else self
  13119         cols = data.columns
  13120         idx = cols.copy()
> 13121         mat = data.to_numpy(dtype=float, na_value=np.nan, copy=False)
  13122
  13123         if method == "pearson":
  13124             correl = libalgos.nancorr(mat, minp=min_periods)

File c:\Users\Utente\AppData\Local\spyder-6\Lib\site-packages\pandas\core\frame.py:2081, in DataFrame.to_numpy(self,
dtype, copy, na_value)
   2077                [2, 4.5, Timestamp('2000-01-02 00:00:00')]], dtype=object)
   2078         """
   2079         if dtype is not None:
   2080             dtype = np.dtype(dtype)
-> 2081         result = self._mgr.as_array(dtype=dtype, copy=copy, na_value=na_value)
   2082         if result.dtype is not dtype:
   2083             result = np.asarray(result, dtype=dtype)
   2084

File c:\Users\Utente\AppData\Local\spyder-6\Lib\site-packages\pandas\core\internals\managers.py:1864, in
BlockManager.as_array(self, dtype, copy, na_value)
   1862         arr.flags.writeable = False
   1863 else:
-> 1864     arr = self._interleave(dtype=dtype, na_value=na_value)
   1865     # The underlying data was copied within _interleave, so no need
   1866     # to further copy if copy=True or setting na_value
   1868 if na_value is lib.no_default:

File c:\Users\Utente\AppData\Local\spyder-6\Lib\site-packages\pandas\core\internals\managers.py:1917, in
BlockManager._interleave(self, dtype, na_value)
   1911 rl = blk.mgr_locs
   1912 if blk.is_extension:
   1913     # Avoid implicit conversion of extension blocks to object
   1914
   1915     # error: Item "ndarray" of "Union[ndarray, ExtensionArray]" has no
   1916     # attribute "to_numpy"
-> 1917     arr = blk.values.to_numpy(  # type: ignore[union-attr]
   1918         dtype=dtype,
   1919         na_value=na_value,
   1920     )
   1921 else:
   1922     arr = blk.get_values(dtype)

File c:\Users\Utente\AppData\Local\spyder-6\Lib\site-packages\pandas\core\arrays\arrow\array.py:1717, in
ArrowExtensionArray.to_numpy(self, dtype, copy, na_value)
   1715 result = data._pa_array.to_numpy()
   1716 if dtype is not None:
-> 1717     result = result.astype(dtype, copy=False)
   1718 if copy:
   1719     result = result.copy()

ValueError: could not convert string to float: ' Male'
```

```
df.select_dtypes('number')
```

	Income	Limit	Rating	Cards	Age	Education	Balance
0	14.891	3606	283	2	34	11	333
1	106.025	6645	483	3	82	15	903
2	104.593	7075	514	4	71	11	580
3	148.924	9504	681	3	36	11	964
4	55.882	4897	357	2	68	16	331
...	...	...	...	...	...	...	...
395	12.096	4100	307	3	32	13	560
396	13.364	3838	296	5	65	17	480
397	57.872	4171	321	5	67	12	138
398	37.728	2525	192	1	44	13	0
399	18.701	5524	415	5	64	7	966

400 rows × 7 columns

```
df.select_dtypes('object')
```

```
C:\Users\Utente\AppData\Local\Temp\ipykernel_2556\1231728108.py:1: Pandas4Warning: For backward compatibility, 'str' dtypes
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to
  df.select_dtypes('object')
```

	Gender	Student	Married	Ethnicity
0	Male	No	Yes	Caucasian
1	Female	Yes	Yes	Asian
2	Male	No	No	Asian
3	Female	No	No	Asian
4	Male	No	Yes	Caucasian
...	...	...	...	...
395	Male	No	Yes	Caucasian
396	Male	No	No	African American
397	Female	No	Yes	Caucasian
398	Male	No	Yes	Caucasian
399	Female	No	No	Asian

400 rows × 4 columns

```
round(df.select_dtypes('number').corr(),2)
```

	Income	Limit	Rating	Cards	Age	Education	Balance
Income	1.00	0.79	0.79	-0.02	0.18	-0.03	0.46
Limit	0.79	1.00	1.00	0.01	0.10	-0.02	0.86
Rating	0.79	1.00	1.00	0.05	0.10	-0.03	0.86
Cards	-0.02	0.01	0.05	1.00	0.04	-0.05	0.09
Age	0.18	0.10	0.10	0.04	1.00	0.00	0.00
Education	-0.03	-0.02	-0.03	-0.05	0.00	1.00	-0.01
Balance	0.46	0.86	0.86	0.09	0.00	-0.01	1.00

La matrice di correlazione è quadrata e simmetrica. Sulla diagonale principale ci sono tutti 1 -> la correlazione infatti è un coefficiente tra 0 e 1 (se positiva) e tra 0 e -1 (se negativa).

Vogliamo estrarre solo la matrice traingolare superiore.

```
import numpy as np    #  un altro package di python fondamentale;
                      #  serve per molti calcoli numerici;
                      #  per convenzione è importato come 'np'

correlation_matrix = df.select_dtypes('number').corr()
upper_triangular_matrix = np.triu(correlation_matrix)
display(upper_triangular_matrix)   # numpy lavora internamente con le ARRAY (numeriche) e non i dataframe
                                   # quindi restituisce una array (una matrice numerica) SENZApiùi nomi delle colonne
mat_corr_upper = pd.DataFrame(upper_triangular_matrix, columns=df.select_dtypes('number').columns)
                                   # 'pd.DataFrame' è una FUNZIONE con la lista degli argomenti tra ();
                                   # invece 'select_dtypes' e 'columns' sono 2 METODI della classe DataFrame e si scrivono
display(round(mat_corr_upper,2))
```

```
array([[ 1.        ,  0.79208834,  0.79137763, -0.01827261,  0.1753384 ,
        -0.02769198,  0.46365646],
       [ 0.        ,  1.        ,  0.99687974,  0.01023133,  0.10088792,
        -0.02354853,  0.86169727],
       [ 0.        ,  0.        ,  1.        ,  0.05323903,  0.103165  ,
        -0.03013563,  0.86362516],
       [ 0.        ,  0.        ,  0.        ,  1.        ,  0.04294829,
        -0.05108422,  0.08645635],
       [ 0.        ,  0.        ,  0.        ,  0.        ,  1.        ,
         0.00361928,  0.00183512],
       [ 0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
         1.        , -0.00806158],
       [ 0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
         0.        ,  1.        ]])
```

	Income	Limit	Rating	Cards	Age	Education	Balance
0	1.0	0.79	0.79	-0.02	0.18	-0.03	0.46
1	0.0	1.00	1.00	0.01	0.10	-0.02	0.86
2	0.0	0.00	1.00	0.05	0.10	-0.03	0.86
3	0.0	0.00	0.00	1.00	0.04	-0.05	0.09
4	0.0	0.00	0.00	0.00	1.00	0.00	0.00
5	0.0	0.00	0.00	0.00	0.00	1.00	-0.01
6	0.0	0.00	0.00	0.00	0.00	0.00	1.00

La seguente cella è stata generata dall'**assistente AI**: avevamo chiesto come NON visualizzare la matrice triangolare inferiore (neanche gli 0)

```
# Hide the lower triangular part (including the diagonal)
def hide_lower_triangular(val):
    return 'visibility: hidden' if val == 0 else ''

# Apply the styling to the upper triangular matrix
styled_matrix = mat_corr_upper.style.map(hide_lower_triangular)
display(styled_matrix)
```

	Income	Limit	Rating	Cards	Age	Education	Balance
0	1.000000	0.792088	0.791378	-0.018273	0.175338	-0.027692	0.463656
1		1.000000	0.996880	0.010231	0.100888	-0.023549	0.861697
2			1.000000	0.053239	0.103165	-0.030136	0.863625
3				1.000000	0.042948	-0.051084	0.086456
4					1.000000	0.003619	0.001835
5						1.000000	-0.008062
6							1.000000

L'indice di correlazione tra Rating e Limit è altissimo (0.9968). Vuol dire che c'è una fortissima **associazione** tra le due variabili, non necessariamente un rapporto causa effetto. Un esempio migliore è la correlazione tra Balance e Limit (0.86): il limite è deciso dalla banca unicamente sulla base del Balance? Probabilmente, no.

Se ci fosse un rapporto causa-effetto tra due variabili correlate (ad esempio Balance e Limit), le troveremmo correlate in OGNI campione di questa popolazione di questa banca.

In QUESTO campione abbiamo rilevato una semplice ASSOCIAZIONE, che potrebbe benissimo non ripetersi in un altro campione.

In italiano e in inglese: correlazione vuol dire rapporto "causa-effetto". L'uomo della srrada intende così la correlazione.

Rename di un nome colonna

```
df.rename(columns={'Balance': 'Balance_card'}, inplace=True)
df
```

	Income	Limit	Rating	Cards	Age	Education	Gender	Student	Married	Ethnicity	Balance_card
0	14.891	3606	283	2	34	11	Male	No	Yes	Caucasian	333
1	106.025	6645	483	3	82	15	Female	Yes	Yes	Asian	903
2	104.593	7075	514	4	71	11	Male	No	No	Asian	580
3	148.924	9504	681	3	36	11	Female	No	No	Asian	964
4	55.882	4897	357	2	68	16	Male	No	Yes	Caucasian	331
...	...	...	...	...	...	...	...	...	...	...	...
395	12.096	4100	307	3	32	13	Male	No	Yes	Caucasian	560
396	13.364	3838	296	5	65	17	Male	No	No	African American	480
397	57.872	4171	321	5	67	12	Female	No	Yes	Caucasian	138
398	37.728	2525	192	1	44	13	Male	No	Yes	Caucasian	0
399	18.701	5524	415	5	64	7	Female	No	No	Asian	966

400 rows × 11 columns

Campionamento casuale:

```
df.sample(n=10)   # utile per esaminare il dataset
```

	Income	Limit	Rating	Cards	Age	Education	Gender	Student	Married	Ethnicity	Balance_card
73	23.989	4523	338	4	31	15	Male	No	No	Caucasian	601
131	10.726	1568	162	5	46	19	Male	No	Yes	Asian	0
105	12.031	3182	259	2	58	18	Female	No	Yes	Caucasian	210
287	60.449	3098	272	4	69	8	Male	No	Yes	Caucasian	0
16	53.598	3714	286	3	73	17	Female	No	Yes	African American	0
282	63.534	8100	581	2	50	17	Female	No	Yes	Caucasian	1298
50	36.362	5183	376	3	49	15	Male	No	Yes	African American	654
220	44.847	5765	437	3	53	13	Female	Yes	No	Asian	1246
384	20.088	1870	180	3	76	16	Male	No	No	African American	0
141	91.876	6754	483	2	33	10	Male	No	Yes	Caucasian	605

```
df.sample(n=10, random_state=1000) # con seme, per garantire la "riproducibilità dei risultati"
```

	Income	Limit	Rating	Cards	Age	Education	Gender	Student	Married	Ethnicity	Balance_card
220	44.847	5765	437	3	53	13	Female	Yes	No	Asian	1246
4	55.882	4897	357	2	68	16	Male	No	Yes	Caucasian	331
230	33.214	5137	387	3	59	9	Male	No	No	African American	661
55	32.916	1786	154	2	60	8	Female	No	Yes	Asian	0
312	27.825	5227	386	6	63	11	Male	No	Yes	Caucasian	823
122	62.602	7056	481	1	84	11	Female	No	No	Caucasian	904
281	36.142	1852	183	3	33	13	Female	No	No	African American	0
371	39.116	2150	173	4	75	15	Male	No	No	Caucasian	0
365	113.772	6442	489	4	69	15	Male	Yes	Yes	Caucasian	790
32	134.181	7838	563	2	48	13	Female	No	No	Caucasian	526

Shuffling = rimescolamento (NON per serie temporali).
E' buona norma RIMESCOLARE un dataframe prima di applicare ad esso un modello di Machine Learning, poichè il dataframe potrebbe essere ORDINATO rispetto a qualche colonna, e cioò può essere un problema per alcuni modelli.
L'ispezione manuale (visiva) degli eventuali ordinamenti è possibile solo se il dataframe ha poche colonne.

```
n = df.shape[0]                          # il numero di righe del dataframe
df_sample = df.sample(n,random_state=1)  # il trucco è qua, il campionamento è fatto su TUTTE le righe,
                                         # SENZA RI-IMMISSIONE del cliente estratto nell'urna (il dataframe),
                                         #  e quindi costituisce uno shuffling

display(df_sample)
```

	Income	Limit	Rating	Cards	Age	Education	Gender	Student	Married	Ethnicity	Balance_card
398	37.728	2525	192	1	44	13	Male	No	Yes	Caucasian	0
125	27.578	2531	195	1	34	15	Female	No	Yes	Caucasian	0
328	41.192	3673	297	3	54	16	Female	No	Yes	Caucasian	121
339	149.316	10278	707	1	80	16	Male	No	No	African American	1107
172	76.348	4697	344	4	60	18	Male	No	No	Asian	108
...	...	...	...	...	...	...	...	...	...	...	...
255	58.063	4221	304	3	50	8	Male	No	No	African American	118
72	22.939	4923	355	1	47	18	Female	No	Yes	Asian	663
396	13.364	3838	296	5	65	17	Male	No	No	African American	480
235	10.503	2923	232	3	25	18	Female	No	Yes	African American	191
37	30.007	6481	462	2	69	9	Female	No	Yes	Caucasian	1093

400 rows × 11 columns

Tabelle di contingenza (**frequenze**)

```
# df['Gender'].value_counts
df[['Gender']].value_counts()
```

```
Gender
Female    207
 Male     193
Name: count, dtype: int64
```

Vediamo ora la **cross-reference**, cioè la tabella degli incroci tra due colonne (anche detta **tabella di conmtingenza a 2 fattori**.)

```
# questo NON è un METODO ma è una FUNZIONE
# un metodo si può applicare solo agli oggetti (cioè variabili) di una CERTA CLASSE,
# il concetto di funzione è più ampio e meno restrittivo
# 'pd.crosstab' è una FUNZIONE
pd.crosstab(index=df['Gender'],columns=df['Married'])
```

Married	No	Yes
Gender		
Male	76	117
Female	79	128

Gli **indici**:

```
df.index         # quello CORRENTE, non necessariamente quello INIZIALE creato automaticamente
                 # da pandas (un progressivo da 0 in poi)
                 # l'indice serve a velocizzare gli accessi.
```

```
RangeIndex(start=0, stop=400, step=1)
```

Possiamo modificare l'indice, ad esempio per la colonna 'Education'

```
df.set_index('Education',inplace=True)
df
```

Education	Income	Limit	Rating	Cards	Age	Gender	Student	Married	Ethnicity	Balance_card
11	14.891	3606	283	2	34	Male	No	Yes	Caucasian	333
15	106.025	6645	483	3	82	Female	Yes	Yes	Asian	903
11	104.593	7075	514	4	71	Male	No	No	Asian	580
11	148.924	9504	681	3	36	Female	No	No	Asian	964
16	55.882	4897	357	2	68	Male	No	Yes	Caucasian	331
...	...	...	...	...	...	...	...	...	...	...
13	12.096	4100	307	3	32	Male	No	Yes	Caucasian	560
17	13.364	3838	296	5	65	Male	No	No	African American	480
12	57.872	4171	321	5	67	Female	No	Yes	Caucasian	138
13	37.728	2525	192	1	44	Male	No	Yes	Caucasian	0
7	18.701	5524	415	5	64	Female	No	No	Asian	966

400 rows × 10 columns

```
df.index        # fornisce l'elenco dei valori (tutti e 400!) del NUOVO indice corrente (Education)
```

```
Index([11, 15, 11, 11, 16, 10, 12,  9, 13, 19,
       ...
       15, 15, 10, 13,  8, 13, 17, 12, 13,  7],
      dtype='int64', name='Education', length=400)
```

```
df.index.is_unique   # test booleano sulla unicità dei valori dell'indice corrente
```

```
False
```

```
df.index.unique()    # lista dei valori dell'indice unici
```

```
Index([11, 15, 16, 10, 12, 9, 13, 19, 14, 7, 17, 8, 5, 20, 18, 6], dtype='int64', name='Education')
```

Una colonna indicizzata non è più subsettabile:

```
df['Education'].head()
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File c:\Users\Utente\AppData\Local\spyder-6\Lib\site-packages\pandas\core\indexes\base.py:3641, in Index.get_loc(self, key)
   3640 try:
-> 3641     return self._engine.get_loc(casted_key)
   3642 except KeyError as err:

File pandas/_libs/index.pyx:168, in pandas._libs.index.IndexEngine.get_loc()
--> 168 'Could not get source, probably due dynamically evaluated source code.'

File pandas/_libs/index.pyx:197, in pandas._libs.index.IndexEngine.get_loc()
--> 197 'Could not get source, probably due dynamically evaluated source code.'

File pandas/_libs/hashtable_class_helper.pxi:7668, in pandas._libs.hashtable.PyObjectHashTable.get_item()
-> 7668 'Could not get source, probably due dynamically evaluated source code.'

File pandas/_libs/hashtable_class_helper.pxi:7676, in pandas._libs.hashtable.PyObjectHashTable.get_item()
-> 7676 'Could not get source, probably due dynamically evaluated source code.'

KeyError: 'Education'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[45], line 1
----> 1 df['Education'].head()

File c:\Users\Utente\AppData\Local\spyder-6\Lib\site-packages\pandas\core\frame.py:4378, in DataFrame.__getitem__(self,
key)
   4374
   4375         if is_single_key:
   4376             if self.columns.nlevels > 1:
   4377                 return self._getitem_multilevel(key)
-> 4378             indexer = self.columns.get_loc(key)
   4379             if is_integer(indexer):
   4380                 indexer = [indexer]
   4381         else:

File c:\Users\Utente\AppData\Local\spyder-6\Lib\site-packages\pandas\core\indexes\base.py:3648, in Index.get_loc(self, key)
   3643     if isinstance(casted_key, slice) or (
   3644         isinstance(casted_key, abc.Iterable)
   3645         and any(isinstance(x, slice) for x in casted_key)
   3646     ):
   3647         raise InvalidIndexError(key) from err
-> 3648     raise KeyError(key) from err
   3649 except TypeError:
   3650     # If we have a listlike key, _check_indexing_error will raise
   3651     #  InvalidIndexError. Otherwise we fall through and re-raise
   3652     #  the TypeError.
   3653     self._check_indexing_error(key)

KeyError: 'Education'
```

Il bottone "Spiega errore" ha fornito la seguente risposta:

> *L'errore KeyError: 'Education' nella cella corrente si è verificato perché la colonna 'Education' è stata impostata come indice del DataFrame utilizzando **df.set_index('Education', inplace=True)**. Una volta che una colonna diventa l'indice, non è più accessibile come una normale colonna utilizzando la sintassi standard df['nome_colonna'].*

> *Per accedere all'indice, puoi usare **df.index**. Per visualizzare i primi valori dell'indice, puoi usare **df.index.head()**.*

> *Se hai bisogno di accedere nuovamente a 'Education' come una normale colonna, puoi resettare l'indice usando **df.reset_index(inplace=True)**.*

```
df.index.head()      # suggerito da Gemini sopra --> un'allucinazione!
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[46], line 1
----> 1 df.index.head()      # suggerito da Gemini sopra --> un'allucinazione!

AttributeError: 'Index' object has no attribute 'head'
```

Uno dei vantaggi dell'argomento 'inplace=True' (disponibile per molti metodi pandas) è di **evitare la proliferazione inutile di dataframe**. Un'alternativa sarebbe infatti: df2 = df.copy()

```
df2 = df       # non crea una copia, crea solo un secondo puntatore al medesimo dataframe - può creare problemi, nel senso
               # risulta modificato anche 'df2'
```